SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

PYTHON PROGRAMMING ASSIGNMENT REPORT

Course: CSA0823 – PYTHON PROGRAMMING

CO3 ASSIGNMENT 3

GITHUB LINK : https://github.com/Harish-S2026/CSA0823-Python-Programming

Submitted by

HARISH S

Register Number: 192512327

DEPT : ECE

Abstract

This report presents the design, implementation, analysis and evaluation of seven Python programming tasks based on lists, tuples, dictionaries, sets, list comprehensions and nested data structures. The tasks use practical contexts including rainfall monitoring, bus-stop mapping, farmer complaints, elective registration, sensor-data cleaning, railway timetables and contact merging. Each solution uses an appropriate built-in data structure, includes boundary-case testing and discusses time and space complexity.

1. Introduction

The objective of CO3 is to implement Python data structures such as lists, tuples, dictionaries and sets to store, access and manipulate unstructured data effectively, including operations, slicing, mutability, copying and list comprehension. The assignment follows the sequence design, implementation, analysis and evaluation.

2. Software and Execution Details

- Language: Python 3.
- Libraries: Python standard library only; modules used include random, time, copy, sys, re and collections.
- Experimental timings are produced by the scripts using time.perf_counter(). They vary with processor, operating system and Python build; therefore the program prints measurements on the execution machine rather than claiming universal fixed values.

Q1 – Lists and Slicing: Daily Rainfall Records

Problem interpretation

A list stores daily rainfall values in chronological order. The program computes seven-day block totals, identifies the wettest and driest blocks, calculates a seven-day moving average and reports the one-based day numbers on which rainfall is zero.

Implementation approach

weekly_totals() slices the list in blocks of seven and applies sum(). The final block may contain fewer than seven values if the input length is not divisible by seven. max() and min() are applied to enumerated weekly totals so the week number is retained. The moving-average function has both a direct window-sum version and a running-sum version.

Moving-average pseudocode

If fewer than seven readings exist, return an empty result. Compute the sum of the first seven readings and output sum/7. For each subsequent reading, add the new reading and subtract the reading that has left the window. Output the updated sum/7. Continue until the last reading.

Slicing explanations

- readings[7:14] returns elements at indexes 7 through 13, which are the eighth to fourteenth readings.
- readings[-7:] returns the last seven elements; if the list has fewer than seven elements, it returns the entire list.
- readings[::7] returns every seventh element, beginning at index zero.

Why a list and operation costs

A list is suitable because it preserves the order of daily observations, supports indexing and slicing, and can grow as new readings arrive. append() is amortized O(1); insert() at an arbitrary position is O(n) because later elements must shift; pop() at the end is O(1), while pop(i) is O(n) due to shifting; sort() is O(n log n) in the average and worst general case (Python's Timsort is adaptive and can be O(n) on already ordered data).

Complexity and addition counts

For n readings and a fixed window w=7, the naive implementation calculates each window sum independently. It performs w additions for each of n-w+1 windows: 7(n-6), which is O(7n), and therefore O(n) for a fixed window. The running-sum method computes the first sum using 6 additions and then uses one addition and one subtraction for each next window, giving 6+2(n-7) arithmetic updates when $n \geq 7$. Its time is O(n), with O(n) output space. The running-sum approach reduces repeated arithmetic.

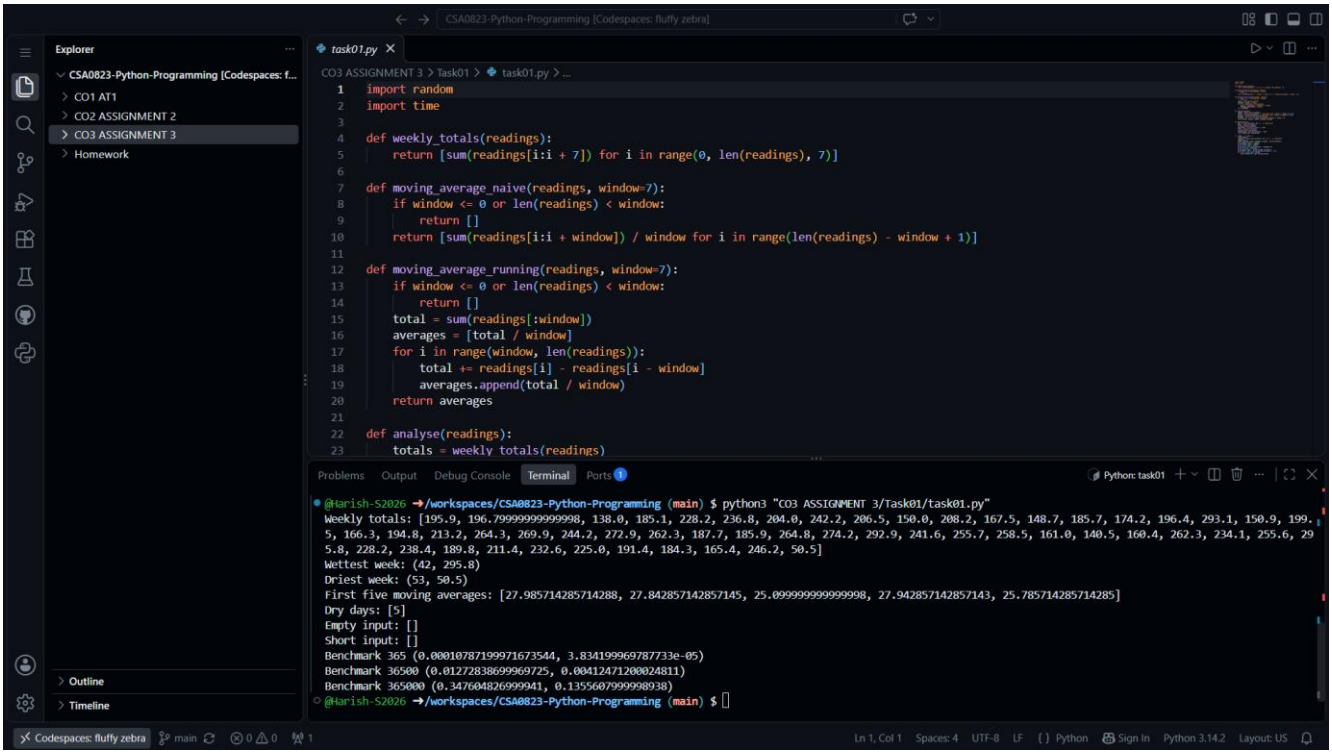
Experimental investigation and edge cases

The source program generates 365, 36,500 and 365,000 random readings and times both methods. It also checks an empty list and a list shorter than seven readings; both correctly produce an empty moving-average result. The generated benchmark values are printed at runtime because measured durations are machine-dependent.

Late-arriving reading and alternative structures

Inserting a late reading into the chronological position of a Python list costs O(n) in the worst case because elements after the insertion point move one position. A list remains reasonable for a 365-day dataset with occasional corrections. For frequent insertions in the middle, a different storage design may be preferable.

A deque(maxlen=7) is useful for a continuously arriving stream: append the new value, automatically discard the oldest value, and update a running sum by subtracting the outgoing value and adding the incoming value. Each update is O(1), and the window storage is O(7)=O(1). A dictionary keyed by date provides average O(1) lookup or update by date, but chronological calculations require sorting dates or maintaining a separate ordered index. A full dictionary uses O(n) storage. A single pass with a running sum takes O(n) time; repeatedly summing seven values takes O(7n).



Q2 – Tuples, Mutability and Copying

Implementation and completed code

GPS coordinates are represented by tuples such as (13.0827, 80.2707). The tuple is immutable and hashable, so it can be used as a dictionary key. The corrected grid is created with good = [[0 for _ in range(cols)] for _ in range(rows)], which creates a distinct inner list for every row. The completed program prints the expected aliased and independent grids, the Boolean identity test True False, and the T. Nagar lookup.

Why tuple keys work and lists do not

Dictionary keys must be hashable and have a stable hash value. A tuple containing only hashable values is hashable, while a list is mutable and therefore unhashable. Attempting stops[[13.0418, 80.2341]] raises TypeError: unhashable type: 'list'.

Aliasing and memory diagram

For bad = [[0] * 3] * 3, the outer list contains three references to the same inner list: bad[0] ─┐ , bad[1] ─┼──> [0, 0, 0], bad[2] ─┘ . Updating one row is visible through all three references. For the comprehension-based grid, each row points to a different list object: grid[0] → [0,0,0], grid[1] → [0,0,0], grid[2] → [0,0,0].

Aliasing assigns another name to the same object. A shallow copy creates a new outer container but retains references to nested objects; list slicing and copy.copy() behave this way. A deep copy recursively copies nested objects. A shallow copy is sufficient when the elements are immutable or

nested objects will not be modified. A deep copy is required when independent nested mutable objects are needed.

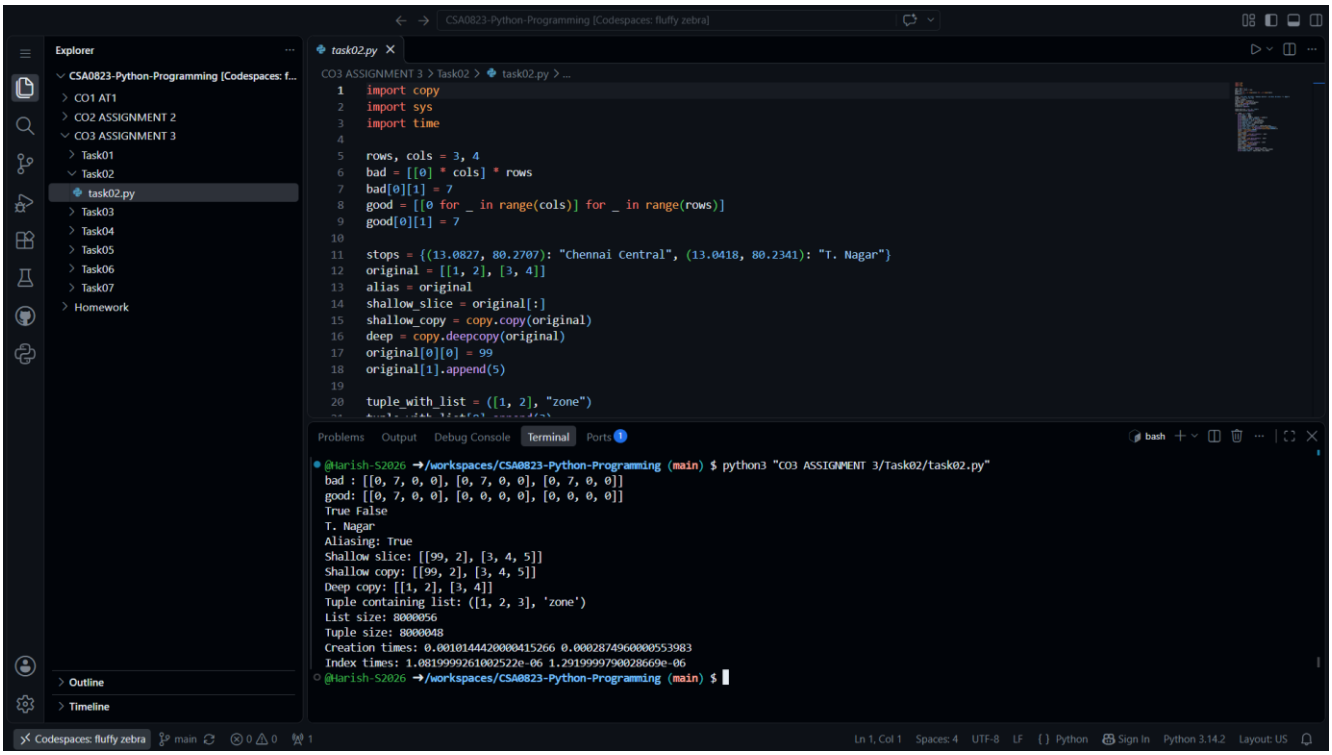
A function that receives a list receives a reference to the same mutable list object. Mutating that object inside the function is therefore visible to the caller. Rebinding the local parameter to a different list does not rebind the caller's variable.

Experimental investigation and tuple containing a list

The program uses is and id-related object identity checks to expose the repeated-row bug, compares sys.getsizeof() for one-million-element list and tuple containers, and times construction and indexing for equal data. Exact byte counts and timings depend on the Python implementation and machine. A tuple can contain a list: the tuple's element reference cannot be replaced, but the referenced list can still be mutated. Thus tuple immutability is shallow, not recursive.

Conclusion and alternative sparse-grid design

Use a tuple for fixed records such as coordinates, where the values should not be reassigned and a hashable key is useful. Immutability protects the tuple's own structure from accidental reassignment and allows stable dictionary-key behaviour. For a sparse passenger grid, a dictionary keyed by (row, column) stores only non-empty zones. Dense list-of-lists access is O(1) and uses O(rows×cols) space. Dictionary access is average O(1), while space is O(k), where k is the number of non-empty zones; it is preferable when k is much smaller than rows×cols.



Q3 – Dictionaries: Farmer Helpline Complaint Analysis

Implementation

The program converts complaint text to lowercase, extracts alphabetic words with a regular expression, removes stop words using a set and counts the remaining words with dict.get(). The top-k function sorts by descending frequency and then ascending alphabetical order. For equal counts, price appears before received because 'price' precedes 'received' alphabetically.

The expected example result is [('seeds', 3), ('price', 2), ('received', 2)]. Grouping by district can be implemented with groups.setdefault(district, []).append(complaint), creating a list only when the district key is first encountered.

Dictionary behaviour and complexity

A dictionary computes a hash for a key and uses it to locate a table position, checking equality when needed to resolve collisions. Lookup, insertion and update are O(1) on average, although pathological collision cases can be slower. The average lookup cost does not grow linearly with the number of stored keys.

Counting with a dictionary processes each token once, O(n) time for n tokens and O(d) space for d distinct words. Calling list.count() once for each distinct word scans the token list repeatedly, O(n·d) time and O(n+d) storage in a straightforward implementation. collections.Counter is a concise frequency-counting alternative and has O(n) expected processing time and O(d) space.

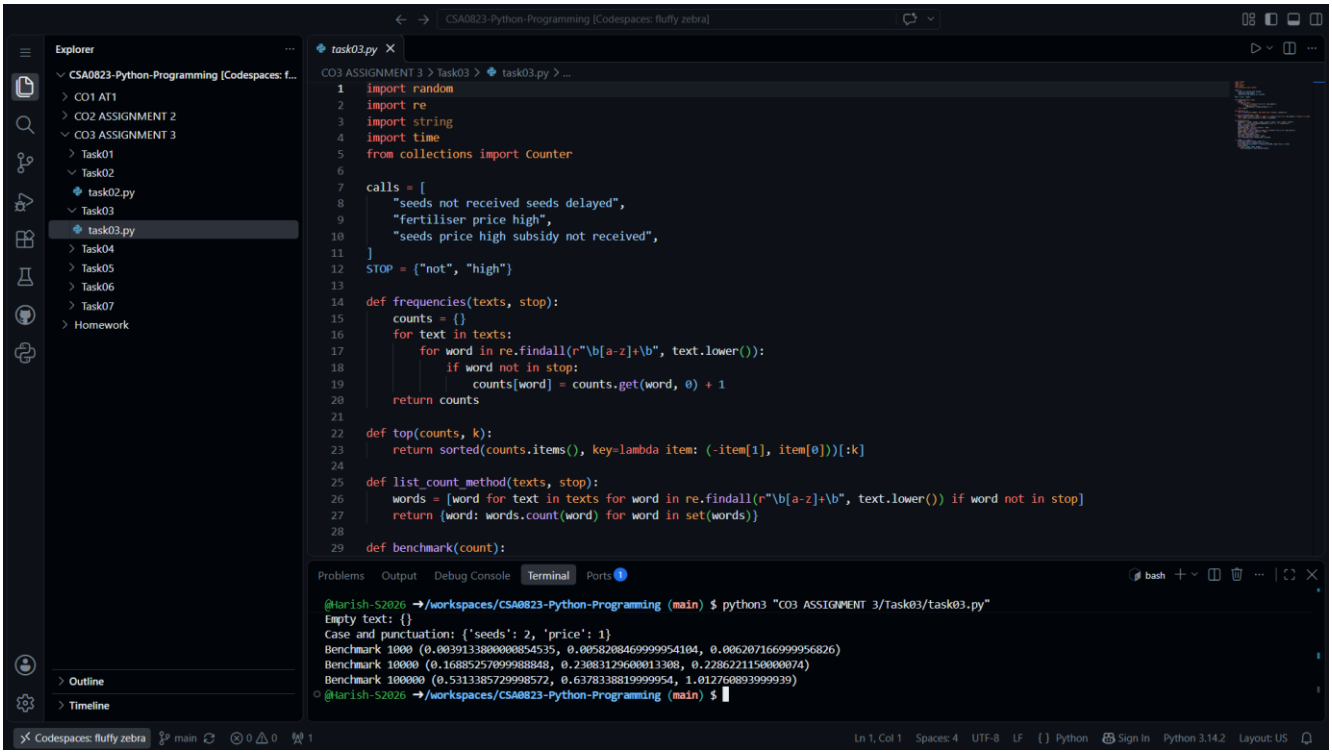
Python 3.7 and later guarantee dictionary insertion order. keys(), values() and items() return dynamic view objects for keys, values and key-value pairs respectively; they reflect later dictionary changes.

Experimental investigation and deployment decision

The script generates 1,000, 10,000 and 100,000 synthetic complaints, then times the dictionary, list.count() and Counter approaches. It also tests empty text, uppercase/lowercase text and punctuation. The benchmark output is generated on execution and should be recorded from the student's own computer. For 100,000 complaints per day, dictionary counting or Counter is the appropriate deployment choice because it scales approximately linearly with the number of tokens; repeated list.count() scans are inefficient.

Alternative analysis

The dictionary solution updates counts as each token is read. Counter performs the same frequency aggregation through a specialised mapping type. A list.count() approach first collects tokens and then scans the full list for each distinct token. The expected time comparison is O(n) versus O(n·d); dictionary and Counter require O(d) count storage, while the token list approach also retains O(n) tokens.



Q4 – Sets: Elective Registration Clash Detection

Implementation and results

Each elective is stored as a set of register numbers, which automatically removes duplicate registrations. The program calculates both = ai_elective & iot_elective, exactly_one = ai_elective ^ iot_elective and neither = all_students - (ai_elective | iot_elective). Sorted output is used because sets do not promise a sorted display order.

Result	Register numbers
Clash / intersection	192311004, 192311009
Exactly one / symmetric difference	192311001, 192311002, 192311007, 192311010
Neither	192311003, 192311005, 192311006, 192311008

Set concepts and Venn interpretation

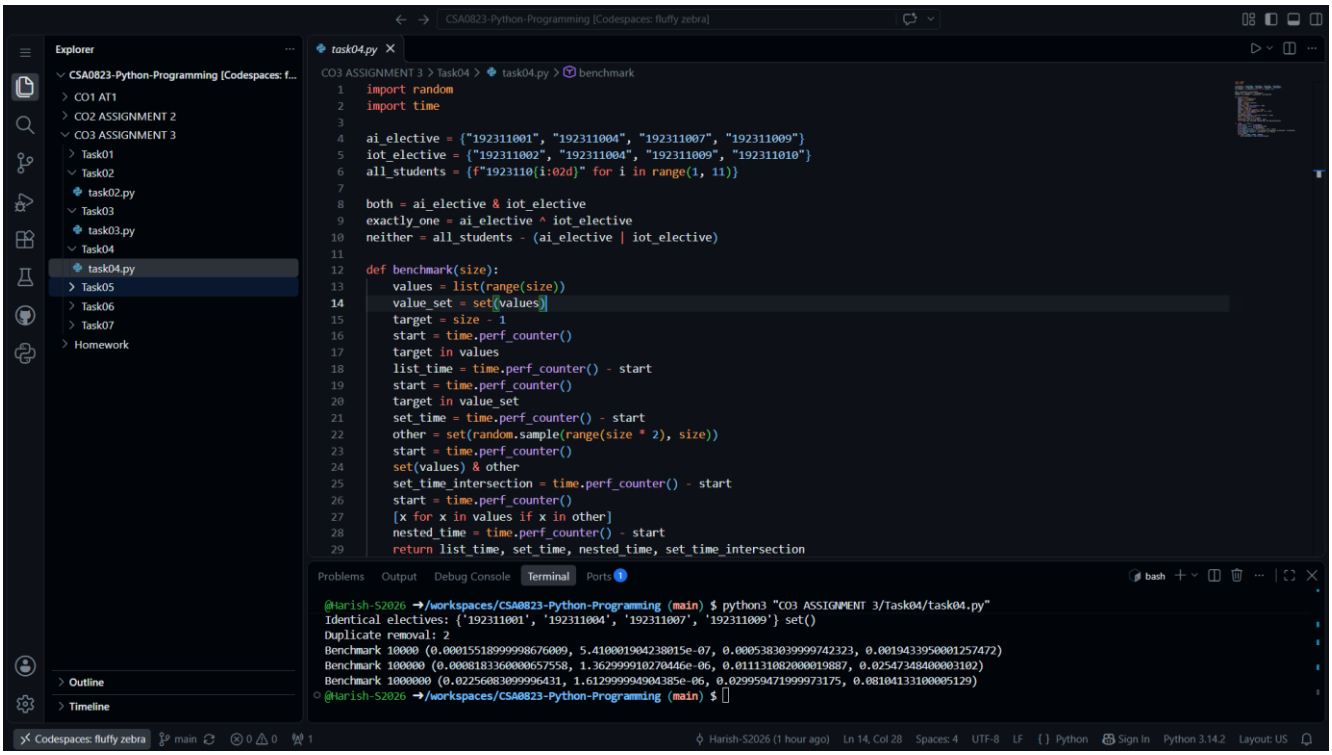
The intersection is the overlapping region of the two circles. The symmetric difference is the two non-overlapping regions. The union is every element in either circle. Neither is the portion of the universe outside the union. Sets remove duplicates because each distinct hashable element can occur only once.

Membership testing in a set is O(1) on average because hashing locates a candidate position; list membership is O(n) because elements may need to be checked one by one. Lists cannot be set elements because they are mutable and unhashable. Tuples containing only hashable elements can be set elements.

Experiments, conclusion and alternative

The script measures membership in lists and sets at 10,000, 100,000 and 1,000,000 values, and compares set intersection with a list-based scan. It also checks empty and identical elective sets and duplicate removal. A set conversion is useful before membership-heavy processing or when unique values are required. Converting to a set discards duplicates and does not preserve the original sequence or positional information.

Nested list comparisons take O(n·m) time for sets of sizes n and m. Sorting both lists and merging them takes O(n log n + m log m) time. Set intersection has O(n+m) expected time in the general size-based analysis and uses additional space for the result. Set methods intersection(), symmetric_difference(), difference() and union() provide the method-based equivalent of &, ^, - and |.



Q5 – List Comprehension: Cleaning Sensor Readings

Implementation

A reading is valid when -50 ≤ temperature ≤ 50. The program produces valid Celsius values, converts them to Fahrenheit using F = (9/5)C + 32, and identifies one-based minute numbers whose temperature is outside the normal 2–8 °C range. It implements the same cleaning operation using a for loop with append(), a list comprehension, and filter()/map() with lambda.

General conditional form: [expression for item in iterable if condition]. A comprehension is concise for a simple transformation or filter, but nested conditions or multiple side effects make a normal loop easier to read. A dictionary comprehension can map minute number to valid temperature; a set comprehension can collect distinct valid readings. A list comprehension creates and stores all results, while a generator expression produces values lazily as they are requested.

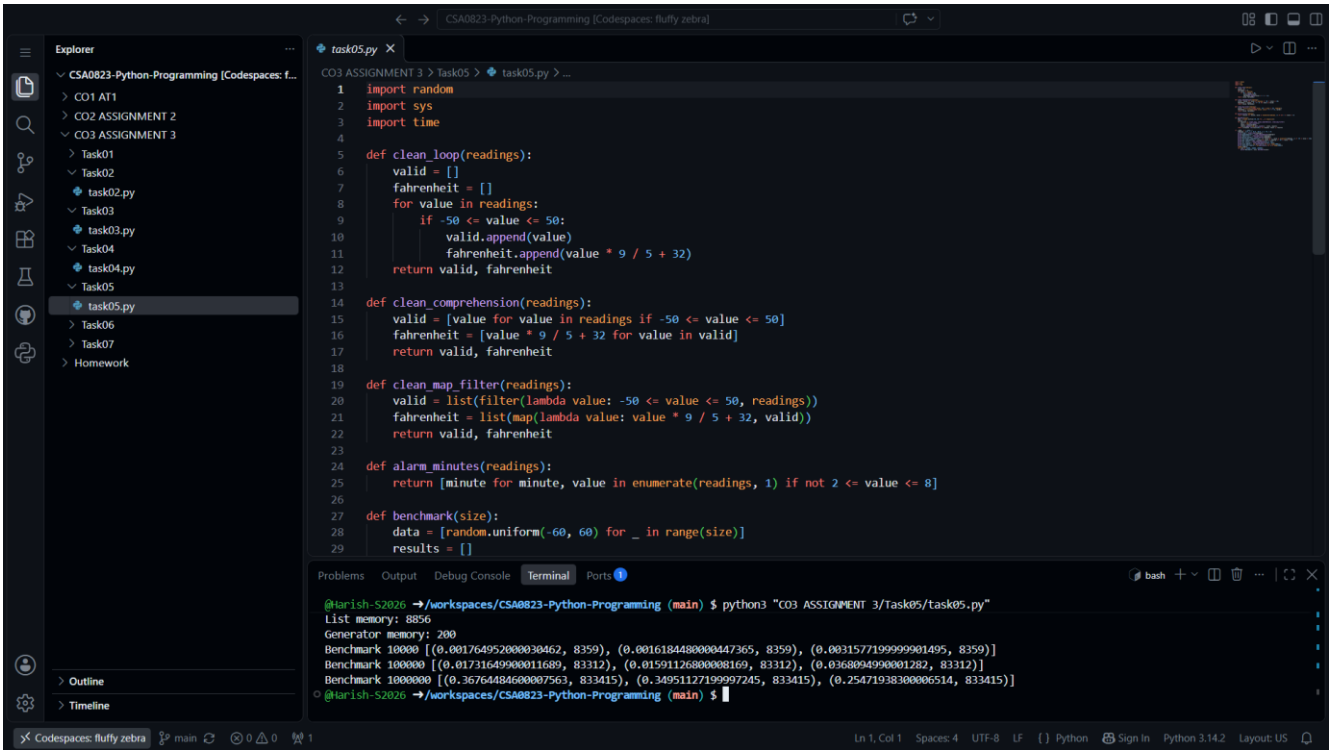
Experiments and complexity

The program times all three methods for 10,000, 100,000 and 1,000,000 readings and verifies their result counts. It also tests all-valid and all-invalid inputs and compares sys.getsizeof() for a list and generator object. A generator's reported object size does not include all memory that may be used by its underlying iterator or source.

All three methods require O(n) time to inspect n readings. The returned lists require O(n) output space. A generator expression itself uses O(1) auxiliary space, excluding the source iterable and any values retained by the consumer. A list comprehension is not always best: use it for clear, compact transformations when the complete result is needed; use a generator when streaming or lower intermediate memory is more important.

Alternative generator solution

A generator such as (c for c in readings if -50 <= c <= 50) yields valid readings one at a time. It can be passed directly to sum(), min() or max() when only an aggregate is needed. Both list and generator approaches inspect O(n) inputs, but the list materialises O(n) results whereas the generator keeps O(1) iteration state.



Q6 – Nested Data: A Railway Timetable

Data model and implementation

The outer dictionary maps a train number to its details. Each details dictionary stores a train name, a list of stop tuples and a set of operating days. A stop tuple has the form (station, arrival, departure). Five sample trains are included. The query checks whether the requested day is in the train's operating-day set and whether the station occurs in its stop list. Matching trains are sorted by departure time using sorted(..., key=lambda item: item[0]).

The departure time at the third stop of train 12601 is timetable['12601']['stops'][2][2]. The nested structure can be read as train number → train details → stops list → third stop tuple → departure field.

Why each data structure is used

- Outer dictionary: direct lookup by unique train number.
- Details dictionary: named fields such as name, stops and days.
- List: preserves the order in which a train visits stations.
- Tuple: groups the fixed station, arrival and departure fields.
- Set: stores unique running days and supports efficient membership tests.

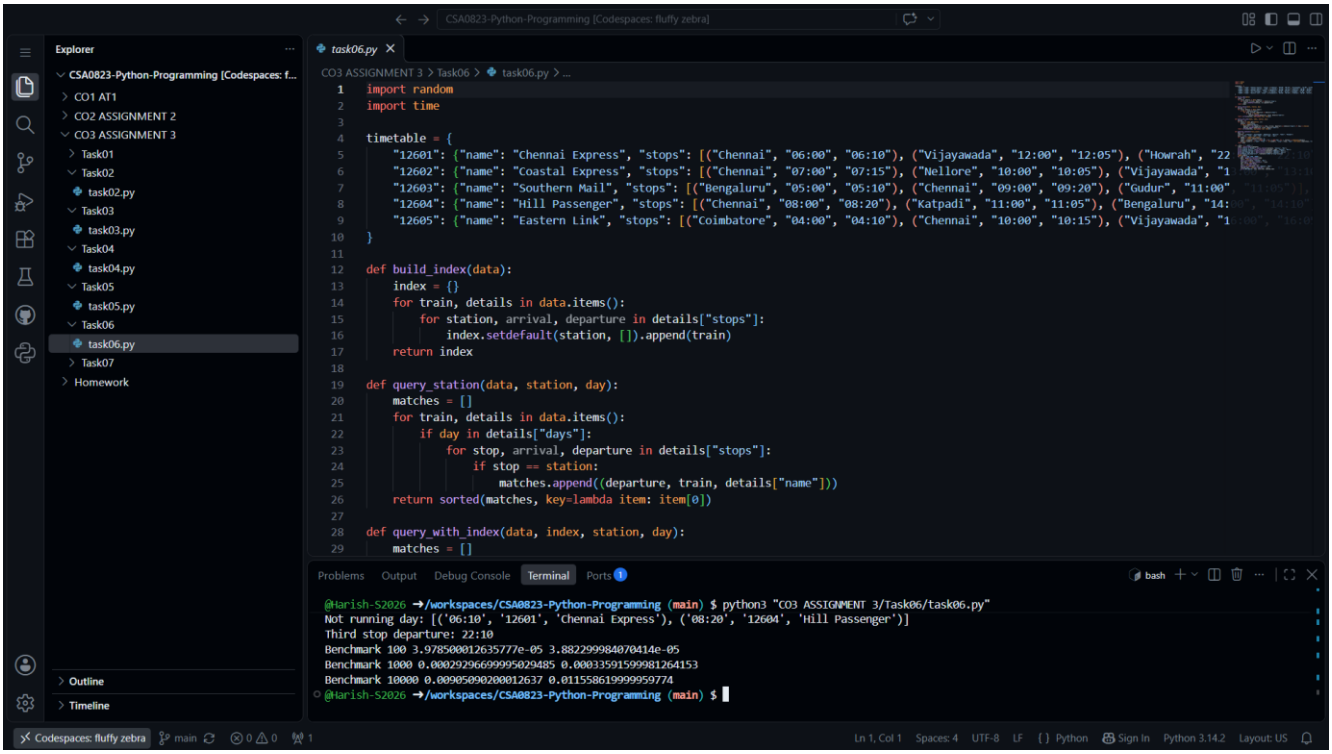
Query complexity and station index

Without an index, a station query scans the trains and their stops, requiring O(S) time where S is the total number of timetable stops, plus sorting of the matches. A station index maps each station to the list of train numbers that stop there. The indexed query examines only the trains in that station's list, approximately O(Ts + matches log matches), where Ts is the number of trains serving the station. The index consumes additional O(S) references in the general case.

Experiments, edge cases and alternative

The program generates timetables of 100, 1,000 and 10,000 trains and times a full scan against an indexed query. It checks an unknown station and a day on which no matching train runs. The measured timings are printed at runtime.

A flat list of (train, station, arrival, departure, days) tuples can be scanned for each query in O(S) time. The nested dictionary supports direct train lookup, while a station index reduces station queries to the number of trains at that station. The trade-off is extra memory and the need to update the index whenever timetable stops change.



Q7 – Design a Contact-Merging Program from First Principles

Stage 1 – Interpret the problem

Inputs are contact records from multiple sources. Each record may contain a name, phone number and email address, and some fields may be missing. Output is a merged collection in which records linked by the same normalised phone number are grouped, with unique phone numbers and email addresses retained.

The primary identity rule is equality of normalised phone number. Phone normalisation removes spaces and punctuation and removes the leading country code 91 when the remaining number has the expected 10-digit Indian national form. If no phone number is available, the implementation uses a normalised name as a fallback key. Measurable requirements are: formatting variants of the same phone merge; duplicate numbers/emails occur once; missing fields do not crash the program; different phone numbers are not merged solely because their names match.

Stage 2 – Design and pseudocode

Normalise a phone by retaining digits only and removing a recognised leading country code. Normalise a name by converting to lowercase, removing punctuation and collapsing whitespace. For each contact, choose the normalised phone as the dictionary key; if absent, use a prefixed normalised-name key. Create a record on first encounter. Add each non-empty normalised phone and email to sets. Return the dictionary values as merged contacts.

Stage 3 – Implementation and testing

The implementation combines two sample contact sources. It tests duplicate entries, different phone formats, missing phone/email fields and case/punctuation variation in names. The two Ravi records merge because +91 98400 12345 and 9840012345 normalise to the same key. Email addresses are stored in a set, so repeated addresses are retained only once. The program benchmarks dictionary grouping at 1,000 and 100,000 contacts; pairwise comparison is measured for the smaller size because its quadratic cost becomes excessive at larger sizes.

Stage 4 – Defend the design

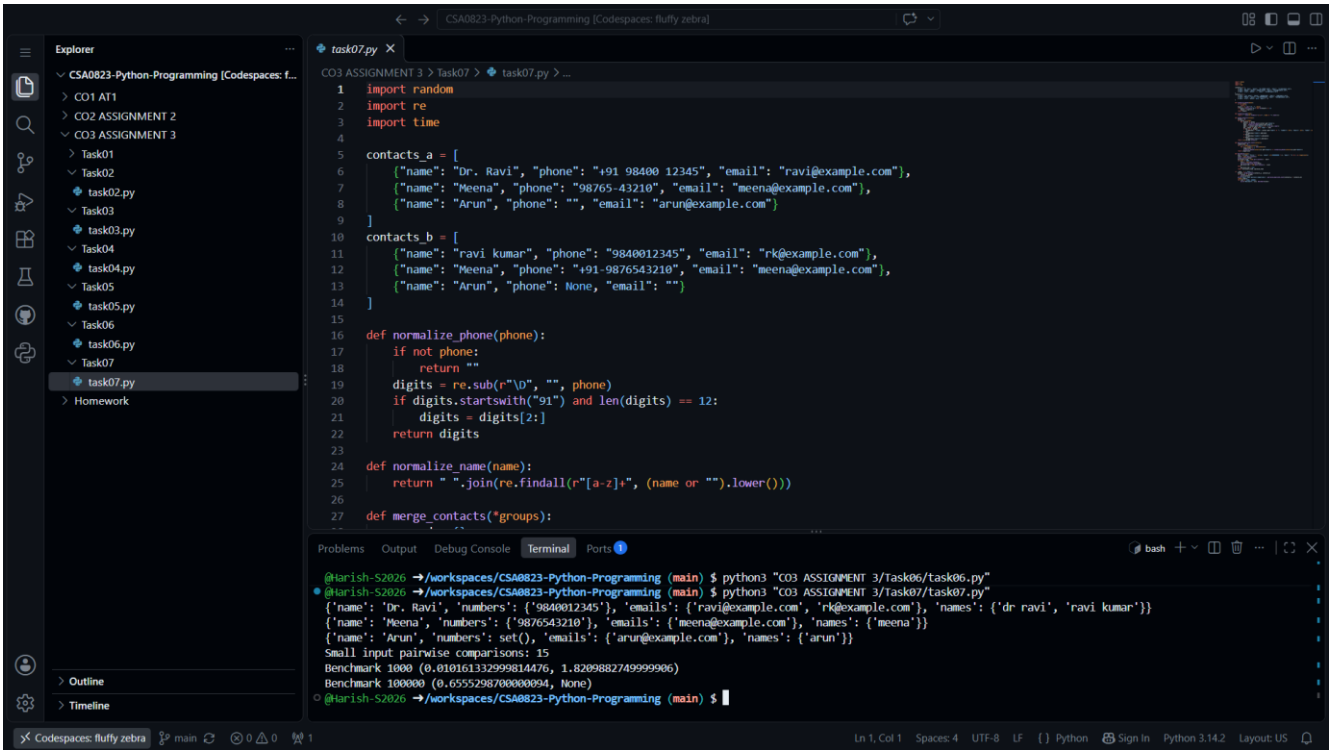
Dictionary grouping is expected $O(n)$ for n contacts because each contact performs an average $O(1)$ dictionary lookup/update. Comparing every pair requires $n(n-1)/2$ comparisons, giving $O(n^2)$ time. The dictionary uses $O(n)$ additional storage for groups and their sets; pairwise comparison can use $O(1)$ auxiliary space if it only checks pairs, though storing merged output also requires $O(n)$ space.

Two people with the same name but different phone numbers remain separate because the phone number is the primary key. A remaining limitation is that a shared family phone number can incorrectly merge different people, while the same person with no phone number and inconsistent name spellings may remain split. A production system would need stronger identity evidence and possibly human review.

Alternative pairwise approach and evaluation

An alternative compares each contact with every previously processed contact, checks normalised phone numbers, and merges matching records. This is simple for small inputs but performs $O(n^2)$ comparisons. Dictionary indexing is preferable for 100,000 contacts because expected $O(n)$ processing

is substantially more scalable. The benchmark function reports the dictionary time for both 1,000 and 100,000 contacts and the pairwise comparison time for 1,000 contacts.



3. Overall Conclusion

The seven tasks demonstrate selecting data structures according to access pattern, mutability, ordering, uniqueness and memory requirements. Lists are appropriate for ordered sequences; tuples represent fixed records and hashable coordinate keys; dictionaries support key-based aggregation and indexing; sets provide uniqueness and efficient membership; comprehensions express compact transformations; and nested structures model hierarchical records.